

OmniScan

Unified Vulnerability Hunting Platform

Build Reference & Architecture Guide

This document is a complete build reference for the OmniScan project — a terminal-based vulnerability orchestration platform that aggregates major scanning engines under one unified TUI interface. Use it as your desk reference throughout development.

Language	Go (primary)
TUI Framework	Bubbletea + Lipgloss + Bubbles
Storage	SQLite (mattn/go-sqlite3)
Build Duration	12 weeks
Scanning Engines	13 tools integrated
Output Formats	HTML, PDF, Markdown, JSON, CSV

Table of Contents

#	Section	Page
1	Core Vision & Overview	3
2	System Architecture	3
3	Technology Stack	4
4	Project Directory Structure	4
5	Phase 1 — Foundation (Weeks 1–2)	5
6	Phase 2 — Scanning Engine Integration (Weeks 3–5)	5
7	Phase 3 — Terminal UI / TUI (Weeks 6–7)	8
8	Phase 4 — Output Normalization (Week 8)	9
9	Phase 5 — Report Generation (Week 9)	11
10	Phase 6 — Bug Bounty Intelligence (Week 10)	11
11	Phase 7 — Polish & Delivery (Weeks 11–12)	12
12	Sample Code: Core TUI Skeleton	12
13	Differentiation Summary	13
14	12-Week Timeline	14
15	Quick Start Checklist	14

1. Core Vision & Overview

OmniScan is a terminal-based vulnerability orchestration platform (TUI) that aggregates every major scanning engine under one unified interface. It does not reinvent scanning — it orchestrates the best tools in existence, normalizes their output, and produces a single consolidated report.

Coverage Areas:

Area	Engine(s)	Scope
CVEs	Nuclei (80,000+ templates), Nmap NSE, OpenVAS	Network & web CVEs
CWEs	Semgrep, Bearer	SAST — maps findings to CWE categories
OWASP Top 10 (2025)	OWASP ZAP, custom checks	All 10 categories
Bug Bounty	FFUF, custom templates	Fuzzing, business logic, recon

2. System Architecture

The platform is layered into four tiers:

Layer	Components	Responsibility
TUI Layer	Bubbletea, Lipgloss, Bubbles	Recon Panel, Scan Panel, Report Viewer, Config Panel
Orchestration Layer	Go goroutines, exec.Command	Manage tools, parallel execution, rate limiting, temp dirs
Output Normalizer	JSON parsers per tool	Deduplication, CWE/CVSS mapping, severity assignment
Scanning Engines	13 external tools	Actual vulnerability detection and recon

Scanning Engines Reference:

Tool	Purpose	Output Format
Nuclei	CVEs, exposures, misconfigs (80k+ templates)	JSON (line-by-line)
OWASP ZAP	DAST — OWASP Top 10:2025	JSON report
OpenVAS	Full network vulnerability assessment	XML
Nmap NSE	Network scanning, service detection	XML
Nikto	Web server vulnerabilities	JSON/CSV
Semgrep	SAST — CWE mapping	JSON
FFUF	Fuzzing, parameter discovery	JSON
Katana / GAU	URL crawling	Text/JSON
Subfinder	Subdomain enumeration	Text
httpx	HTTP probing, tech detection	JSON
TruffleHog	Secret detection	JSON (line-by-line)

3. Technology Stack

Component	Choice	Rationale
TUI	Bubbletea + Lipgloss + Bubbles	Performance, concurrency, used by OpenClaw
Backend	Go (native)	Goroutines for parallel scanning
Database	SQLite (mattn/go-sqlite3)	Lightweight, embedded, portable
Report Generation	Go HTML templates + PDF	Full control over output formatting
Config	YAML / TOML	Human-readable scan profiles

Key Go dependencies:

- github.com/charmbracelet/bubbletea — TUI framework
- github.com/charmbracelet/lipgloss — Terminal styling
- github.com/charmbracelet/bubbles — TUI components (progress, viewport, spinner, table, help)
- [mattn/go-sqlite3](https://github.com/mattn/go-sqlite3) — Embedded SQLite database

4. Project Directory Structure

omniscan/	
cmd/	
omniscan/	<i>Main entry point</i>
main.go	
internal/	<i>All private application code</i>
tui/	<i>Terminal UI components</i>
app.go	
recon_panel.go	
scan_panel.go	
report_panel.go	
styles.go	
scanner/	<i>Orchestration engine</i>
orchestrator.go	
installer.go	
nuclei.go	
zap.go	
nmap.go	
openvas.go	
nikto.go	
semgrep.go	
ffuf.go	
custom.go	
recon/	<i>Reconnaissance modules</i>
subdomain.go	
probing.go	
crawling.go	
normalizer/	<i>Output normalization</i>
parser.go	
dedup.go	
mapper.go	
report/	<i>Report generation</i>
generator.go	
html_template.go	

pdf.go	
db/	Database layer
store.go	
migrations.go	
models.go	
config/	Configuration
config.go	
defaults.go	
pkg/	
types/	Shared types
vuln.go	Unified Finding struct
templates/	Nuclei custom templates
cves/	
exposures/	
bug-bounty/	
web/	Web UI fallback
go.mod	
go.sum	
Makefile	

5. Phase 1 — Foundation (Weeks 1–2)

- Set up Go module: `go mod init github.com/yourname/omniscan`
- Install Bubbletea: `go get github.com/charmbracelet/bubbletea`
- Scaffold the directory structure above
- Build omniscan setup command — auto-installs all 13 tools
- Create parallel installer with TUI progress bar

Tool Installer Skeleton (*internal/scanner/installer.go*):

```
func (i *Installer) InstallAll() map[string]InstallResult { tools := map[string]InstallFunc{ "nuclei":  
installNuclei, "subfinder": installSubfinder, "httpx": installHttpx, "katana": installKatana, "ffuf":  
installFfuf, "nmap": installNmap, "nikto": installNikto, "openvas": installOpenVAS, "zap": installZap,  
"semgrep": installSemgrep, "trufflehog": installTrufflehog, "gau": installGau, "gospider":  
installGospider, } // Parallel install with progress bar }
```

6. Phase 2 — Scanning Engine Integration (Weeks 3–5)

6.1 Orchestrator — Scan Pipeline

The orchestrator runs the pipeline in sequential stages:

Stage	Const Name	Tools Run
0	StageRecon	Subfinder → httpx → tech detection
1	StageCrawling	Katana, GAU — URL collection
2	StageFuzzing	FFUF — parameter discovery
3	StageVulnScan	Nuclei, OpenVAS, Nikto
4	StageDeepScan	ZAP, custom payloads
5	StageSAST	Semgrep (if source code available)
6	StageSecrets	TruffleHog
7	StageReporting	Generate consolidated report

ScanPipeline struct:

```
type ScanPipeline struct { Target string Scope []string // in-scope domains/IPs Stage ScanStage  
Progress float64 Findings []Finding StartTime time.Time }
```

6.2 Nuclei Integration (Primary CVE Engine)

Nuclei is the primary CVE engine with 80,000+ templates. Run with JSON output streamed line-by-line into the TUI parser.

```
args := []string{ "-l", targetsFile, "-t", templateDir, "-severity", "critical,high,medium,low",  
"-rate-limit", "150", "-bulk-size", "25", "-json", "-o", outputFile, "-stats", // real-time stats for  
TUI "-hm", // headless mode for JS-heavy targets } cmd := exec.Command("nuclei", args...) // Stream  
JSON output line-by-line back to TUI
```

6.3 OWASP ZAP (DAST — OWASP Top 10:2025)

Run ZAP in daemon/headless mode with active scan policies covering all 10 categories:

Category	Name
A01	Broken Access Control
A02	Cryptographic Failures

A03	Injection (SQLi, XSS, etc.)
A04	Insecure Design
A05	Security Misconfiguration
A06	Vulnerable & Outdated Components
A07	Identification & Authentication Failures
A08	Software & Data Integrity Failures
A09	Security Logging & Monitoring Failures
A10	Server-Side Request Forgery (SSRF)

6.4 Custom Bug Bounty Checks

Build a custom scanner module with bug-bounty-specific test cases:

- IDOR detection (UUID enumeration, sequential IDs)
- Race condition testing
- 2FA bypass checks
- OAuth misconfiguration
- JWT attacks (none algorithm, key confusion)
- SSTI detection (all template engines)
- GraphQL introspection + batching attacks
- WebSocket vulnerabilities
- Cache poisoning / cache deception
- Prototype pollution (client + server side)
- Host header injection
- CRLF injection
- Account takeover vectors

7. Phase 3 — Terminal UI / TUI (Weeks 6–7)

Built with Bubbletea. The main model contains four panels plus a live log viewport.

7.1 Main Model Structure

```
type mainModel struct { tabs []string activeTab int scanProgress progress.Model spinner spinner.Model
resultsTable table.Model logViewport viewport.Model logs []string help help.Model keys keyMap ready
bool }
```

7.2 Key Bindings

Key	Action
s	Start scan pipeline
e	Export report
Tab / l	Next tab
Shift+Tab / h	Previous tab
Ctrl+C / q	Quit

7.3 TUI Layout (Visual Reference)

The TUI is divided into a title bar, four panels, a live log, and a footer key bar:

OmniScan v1.0		Target: target.com		Status: SCANNING
RECON	SCAN ENGINE	FINDINGS	REPORT	
+ Subfinder	> Nuclei [####]	CRITICAL: 3	[Export]	
+ httpx	> ZAP [####]	HIGH: 12	[PDF] [HTML]	
+ Katana	> Nmap [#####]	MEDIUM: 28	[JSON]	
+ GAU	> FFUF []	LOW: 45		
LIVE LOG				
[15:32:01] [NUCLEI] CVE-2025-4427 found on admin.target.com				
[15:32:03] [ZAP] SQL Injection detected on /api/users?id=1				
[15:32:05] [FFUF] Endpoint found: /admin/config.php (HTTP 200)				
[15:32:07] [NUCLEI] CVE-2025-61882 found on app.target.com				
[Tab: Scan] [Tab: Results] [Tab: Report]			CTRL+C: Stop Scan	

Panel descriptions: RECON shows active discovery tools; SCAN ENGINE shows each tool with a live progress bar; FINDINGS shows real-time severity counts; REPORT shows export options. The LIVE LOG streams raw findings as they arrive from each tool.

8. Phase 4 — Output Normalization (Week 8)

The normalization layer is the hardest part — making every tool speak the same language. Each tool has a dedicated parser that maps its output to the unified Finding struct.

8.1 Unified Finding Struct (pkg/types/vuln.go)

```
type Finding struct { ID string // hash dedup key Title string Description string Severity Severity
CVSS float64 CVE string // optional CWE []string // optional OWASP2025 string // optional AffectedURL
string AffectedParam string // optional Payload string // optional Proof string // optional Remediation
string ToolSource string // nuclei, zap, nmap, etc. Timestamp time.Time CVSSVector string // optional
EPSS float64 // exploit prediction score FalsePositive bool // manual flag BountyPlatforms []string //
optional }
```

8.2 Tool Parsers (internal/normalizer/parser.go)

```
func ParseNucleiOutput(jsonLine []byte) (*Finding, error) { ... } func ParseZapOutput(jsonReport
[]byte) ([]Finding, error) { ... } func ParseNmapOutput(xmlData []byte) ([]Finding, error) { ... } func
ParseNiktoOutput(jsonLine []byte) (*Finding, error) { ... } func ParseSemgrepOutput(jsonData []byte)
([]Finding, error){ ... } func ParseOpenVASOutput(xmlData []byte) ([]Finding, error) { ... } func
ParseTrufflehogOutput(jsonLine []byte) (*Finding, error){ ... }
```

8.3 Deduplication Strategy

Rule	Logic
1	Same URL + same CVE → keep highest severity instance
2	Same URL + same CWE + same param → merge into one finding
3	Same CVE across multiple tools → keep entry with most detail
4	Confidence scoring: Nuclei proof > ZAP heuristic

8.4 CWE → OWASP 2025 Mapping (partial — build full list of 100+ CWEs)

CWE	OWASP 2025 Category
CWE-287, CWE-285, CWE-22	A01 — Broken Access Control
CWE-311, CWE-327	A02 — Cryptographic Failures
CWE-89, CWE-79, CWE-78	A03 — Injection
CWE-200, CWE-209	A05 — Security Misconfiguration
CWE-798, CWE-287	A07 — Identification & Auth Failures
CWE-918	A10 — Server-Side Request Forgery

9. Phase 5 — Report Generation (Week 9)

Output Format Options:

Format	Primary Use Case
HTML	Interactive report with collapsible sections and embedded screenshots
PDF	Professional submission-ready reports for clients
Markdown	Paste directly into HackerOne / Bugcrowd write-ups
JSON	CI/CD pipeline integration and programmatic processing
CSV	Spreadsheets, issue tracking, team hand-offs

Report Sections (in order):

- Cover: Target, scan date, duration, tools used, total vuln count
- Executive Summary: Severity breakdown bar chart, CVSS average, CWE count, OWASP coverage
- Top 5 Critical Findings: CVE/CWE, CVSS, EPSS, affected URL, remediation
- Full Findings Table: sortable by severity, includes tool source
- Appendix A: Full proof, payloads, raw requests and responses
- Appendix B: Raw tool output for each scanner

10. Phase 6 — Bug Bounty Intelligence (Week 10)

Feature	Description
EPSS Scoring	Prioritize findings most likely to be exploited in the wild
Duplicate Detection	Check if CVE was already disclosed on HackerOne/Bugcrowd
P1 Pattern Recognition	Learn which findings convert to bounties historically
Weaponization Check	Flag findings with public PoC — increases urgency
Program Scope Loading	Pull scope from HackerOne/Bugcrowd API automatically
Rate Limiting	Respect per-program rate thresholds to avoid bans
Custom Templates	Auto-update from projectdiscovery/nuclei-templates daily

Template Manager responsibilities:

- Auto-update templates from projectdiscovery/nuclei-templates daily
- Maintain custom templates for zero-day/POC-based vulns
- Tag templates by: cve, cwe, owasp, technology, bugbounty
- Priority scoring based on real-world impact

11. Phase 7 — Polish & Delivery (Weeks 11–12)

Installation Commands:

```
# One command installation curl -fsSL https://omniscan.dev/install.sh | bash # Or via Go go install
github.com/omniscan/cli@latest omniscan setup # Installs all 13 tools # Quick scan omniscan scan -t
target.com # Bug bounty mode omniscan bounty -t target.com -program hackerone # Launch TUI omniscan tui
```

Performance Optimizations:

- Parallel scanning across targets using Go goroutines
- Smart throttling per domain — avoid WAF blocks
- Resume capability — scan picks up from last checkpoint
- DNS result caching and previous scan state caching
- Optional distributed scanning — split targets across machines

Distribution Targets:

- Pre-built binaries for Linux / macOS / Windows
- Docker image with all 13 tools pre-installed
- GitHub Actions integration for scheduled automated scans

12. Sample Code: Core TUI Skeleton

cmd/omniscan/main.go — entry point with Bubbletea:

```
package main import ( "fmt" "os" tea "github.com/charmbracelet/bubbletea"
"github.com/charmbracelet/lipgloss" "github.com/charmbracelet/bubbles/viewport"
"github.com/charmbracelet/bubbles/spinner" "github.com/charmbracelet/bubbles/progress"
"github.com/charmbracelet/bubbles/table" "github.com/charmbracelet/bubbles/help"
"github.com/charmbracelet/bubbles/key" ) var docStyle = lipgloss.NewStyle().Margin(1, 2) type keyMap
struct { Quit key.Binding Scan key.Binding Export key.Binding TabNext key.Binding TabPrev key.Binding }
type mainModel struct { tabs []string activeTab int scanProgress progress.Model spinner spinner.Model
resultsTable table.Model logViewport viewport.Model logs []string help help.Model keys keyMap ready
bool } func (m mainModel) Init() tea.Cmd { return tea.Batch(spinner.Tick, m.scanProgress.Init()) } func
(m mainModel) Update(msg tea.Msg) (tea.Model, tea.Cmd) { switch msg := msg.(type) { case tea.KeyMsg:
switch { case key.Matches(msg, m.keys.Quit): return m, tea.Quit case key.Matches(msg, m.keys.TabNext):
m.activeTab = min(m.activeTab+1, len(m.tabs)-1) case key.Matches(msg, m.keys.TabPrev): m.activeTab =
max(m.activeTab-1, 0) case key.Matches(msg, m.keys.Scan): return m, m.startScan() } } return m, nil }
func (m mainModel) View() string { return docStyle.Render(m.renderTabs() + "\n" + m.renderContent()) }
func main() { p := tea.NewProgram(mainModel{ tabs: []string{"Scan", "Results", "Report", "Config"},
scanProgress: progress.New(progress.WithDefaultGradient()), spinner: spinner.New(), help: help.New(),
keys: keyMap{ Quit: key.NewBinding( key.WithKeys("ctrl+c", "q"), key.WithHelp("ctrl+c/q", "quit")), Scan:
key.NewBinding( key.WithKeys("s"), key.WithHelp("s", "start scan")), Export: key.NewBinding(
key.WithKeys("e"), key.WithHelp("e", "export report")), TabNext: key.NewBinding(
key.WithKeys("tab", "l"), key.WithHelp("tab/l", "next tab")), TabPrev: key.NewBinding(
key.WithKeys("shift+tab", "h"), key.WithHelp("S-tab/h", "prev tab")), }, }) if _, err := p.Run(); err !=
nil { fmt.Fprintf(os.Stderr, "Error: %v\n", err) os.Exit(1) } }
```

13. Differentiation Summary

Feature	What It Does
Unified coverage	CVE, CWE, OWASP, SAST, secrets, fuzzing — one tool, one command
Real-time TUI	Watch findings appear as they are discovered, not after scan completes
Dedup engine	Same vuln found by 3 tools = 1 report entry (with confidence scoring)
CVE→CWE→OWASP mapping	Automatic classification across all three taxonomies
Bug bounty intelligence	EPSS scoring, duplicate checking, program-aware rate limiting
Submission-ready reports	PDF/HTML reports formatted for HackerOne and Bugcrowd
Custom template ecosystem	Write YAML templates for new zero-days and share them
Resume & partial scan	Kill and resume without losing any progress
Parallel execution	All tools run concurrently — full scan in minutes not hours

14. 12-Week Timeline

Week(s)	Deliverable
1–2	Project scaffolding, Go modules, TUI skeleton, tool installer (omniscan setup)
3–4	Nuclei + Subfinder + httpx integration, live TUI updates with streaming JSON
5–6	ZAP, Nmap, OpenVAS, FFUF integration
7	Semgrep, TruffleHog, Nikto, Katana integration
8	Output normalizer + dedup engine + CWE/OWASP mapper
9	Report generator — HTML, PDF, Markdown, JSON
10	Bug bounty intelligence: EPSS scoring, duplicate detection, scope loading
11	Performance optimizations, resume/caching, Docker image
12	Polish, documentation, test suite, first public release

Note: The Nuclei + Subfinder + httpx combo delivers ~80% of total value. All other tools are additive. Prioritize this trio in weeks 3–4.

15. Quick Start Checklist

Use this checklist when you first sit down to build:

- 01. Create Git repository and run: `go mod init github.com/yourname/omniscan`
- 02. Install core TUI dependencies: `go get github.com/charmbracelet/bubbletea lipgloss bubbles`
- 03. Scaffold directory structure (Section 4)
- 04. Build `main.go` TUI skeleton (Section 12)
- 05. Implement omniscan setup installer for all 13 tools
- 06. Wire Subfinder → httpx → stdout pipeline
- 07. Add Nuclei with JSON streaming into TUI live log
- 08. Build the unified Finding struct (Section 8.1)
- 09. Write `ParseNucleiOutput()` parser
- 10. Add progress bars per tool to scan panel
- 11. Integrate ZAP daemon mode
- 12. Integrate FFUF with parameter wordlist
- 13. Write deduplication engine (Section 8.3)
- 14. Build CWE → OWASP mapper (Section 8.4)
- 15. Build HTML report template
- 16. Add PDF export (`go-wkhtmltopdf` or `chromedp` headless)
- 17. Add EPSS API integration for scoring
- 18. Wire remaining 8 tools (Nmap, OpenVAS, Nikto, Semgrep, TruffleHog, Katana, GAU, Gospider)
- 19. Add scan resume / checkpoint state to SQLite
- 20. Build Docker image with all tools pre-installed
- 21. Write README.md and usage docs
- 22. Tag v1.0.0 release and publish binaries

End of OmniScan Build Reference — Good luck with the build.